

SIMATS ENGINEERING

SCENARIO-BASED ASSESSMENT

COURSE NAME	COURSE CODE
Introduction to Large Language Models (LLMs)	AI / LLM

Assessment: Build a complete Retrieval-Augmented Generation (RAG) pipeline.

Question Attempted: Q13 - Farmer Advisory Agent

Total Marks: 100

Code of Conduct:

I _G SAI TEJA (192472137) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

1. Problem Statement

Q13 - Farmer Advisory Agent.

An agri-portal answers from government crop advisories. Task: Build a RAG system over public crop advisory documents. Demonstrate queries on pest control and sowing seasons, and one query in simple non-technical phrasing that must still retrieve correctly.

Assessment requirements addressed:

- Complete RAG pipeline: document loading -> chunking -> embedding -> vector store indexing -> retrieval -> LLM answer generation.
- Corpus of at least 10 pages or 20 documents (this submission uses 20 documents, ~11 pages).
- Answers grounded in retrieved context - the system displays the retrieved chunk(s) used for each answer and cites them as [C#].
- At least five queries: three answerable, one needing multiple chunks, and one out-of-scope query that the system refuses instead of hallucinating.
- Agent behaviour demonstrated: query rewriting (tool use), routing/guardrail, and multi-step grounded reasoning.
- Recorded design choices, an architecture diagram, and justification of each decision.

2. Solution Overview

The Farmer Advisory Agent is a self-contained RAG system. At startup it loads 20 crop advisory documents, splits them into overlapping chunks, embeds each chunk with a from-scratch TF-IDF vectoriser, and stores the vectors in an in-memory cosine index. At query time an agent first rewrites the farmer's plain-language question into technical terms, retrieves the top-3 most similar chunks, decides whether the question is in scope, and then generates a short, grounded answer that cites the exact chunks it used. If the language model is unavailable, the system falls back to an extractive answer built from the same retrieved chunks, so it is always grounded and never fails during a demo.

Implementation: a single Python file, `lab/q13_farmer_rag.py`, with no heavy dependencies (only `httpx` + `python-dotenv`, already used by the main app). The LLM is the same OpenAI-compatible free model the project already uses (`nvidia/nemotron-nano-9b-v2:free` via OpenRouter).

3. Architecture Diagram

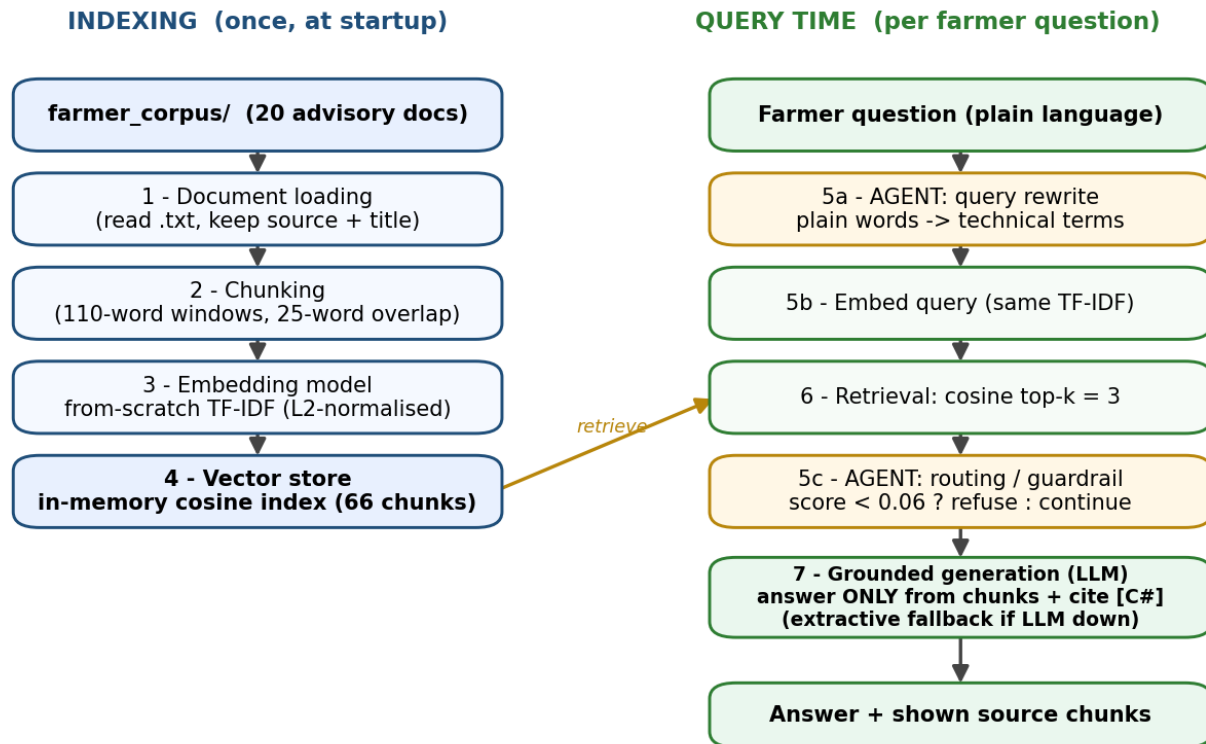


Figure 1. The RAG pipeline. Blue = indexing stages, green = query-time stages, amber = agent decision steps.

The seven pipeline stages:

- Document loading - read every .txt advisory, keep its source filename and title.
- Chunking - split each document into 110-word windows with 25-word overlap (66 chunks total).
- Embedding - convert each chunk into an L2-normalised TF-IDF vector (the embedding model).
- Vector store indexing - keep all chunk vectors in an in-memory cosine index.
- Retrieval - embed the query the same way and take the top-3 chunks by cosine similarity.
- Agent - rewrite the query before retrieval and route (answer vs refuse) after retrieval.
- Generation - the LLM writes a short answer using ONLY the retrieved chunks and cites them as [C#].

4. Design Choices and Justification

Stage	Choice	Justification
Corpus	20 crop advisory .txt documents (~5,400 words ~ 11 pages)	Meets the 10-page or 20-document minimum on both counts; covers 15+ crops so pest-control and sowing-season questions are both answerable. Compiled in the style of public agricultural-extension advisories as a reasonable substitute.
Chunking	110 words per chunk, 25-word overlap (~23%)	Each advisory has short labelled sections (Season, a named pest, Key Points). ~110 words keeps one topic per chunk for precise retrieval; the overlap prevents a fact being split across a boundary.
Embedding model	From-scratch TF-IDF (sublinear tf, smoothed idf, L2 norm)	Zero heavy dependencies -> always runs, fully reproducible and inspectable. Advisories are keyword-rich (crop + pest + season), exactly where TF-IDF excels. idf

		up-weights the discriminating crop/pest names.
Vector store	In-memory cosine index (sparse dict vectors)	66 chunks is tiny; an in-memory index gives exact top-k instantly with no external service (FAISS/Chroma/pgvector) to install. Cosine on normalised vectors = a dot product.
Retrieval (top-k)	k = 3	Enough context for multi-fact and cross-document answers without flooding the prompt with unrelated text that could cause drift.
Routing threshold	Refuse if best cosine < 0.06	Turns the retrieval score into a scope check. In-scope queries score 0.23-0.37; out-of-scope ('gold price') scores 0.000 - a clean margin - so the system refuses instead of hallucinating.
Query rewriting	Plain -> technical synonym expansion	Farmers do not use textbook words. 'bugs/worms/holes/when to plant' map to 'pest/borer/feeding damage/sowing season' before embedding, so non-technical phrasing still lands on the right chunk.
LLM generator	nvidia/nemotron-nano-9b-v2:free, temperature 0.2	Same reliable free model as the main app. Low temperature keeps answers faithful to context. Extractive fallback keeps the demo working if the API is down or rate-limited.

Prompt template (verbatim):

SYSTEM: You are a Farmer Advisory Agent. Answer the farmer's question ONLY using the numbered context passages provided. Cite every passage you use inline with its tag, e.g. [C1], [C2]. Give short, practical, plain-language advice. If the answer is not present in the context, reply exactly:
'I could not find this in the crop advisory documents.'

USER: Context passages:
{numbered retrieved chunks with source + score}

Farmer's question: {question}

Grounded answer with citations:

5. Knowledge Base (Corpus)

20 documents in lab/farmer_corpus/. Each follows the same format: TITLE, CROP, recommended season, named pests/diseases with symptoms and IPM/chemical control, and a Key Points summary.

File	Crop	Topic
01_paddy_sowing_season.txt	Paddy / Rice (<i>Oryza sativa</i>)	Paddy (Rice) — Sowing Season and Nursery Advisory
02_paddy_pest_control.txt	Paddy / Rice	Paddy (Rice) — Pest and Insect Control Advisory
03_wheat_sowing_season.txt	Wheat (<i>Triticum aestivum</i>)	Wheat — Sowing Season and Seed Rate Advisory
04_wheat_pest_disease.txt	Wheat	Wheat — Pest and Disease Control Advisory
05_cotton_sowing_and_bollworm.txt	Cotton (<i>Gossypium</i>)	Cotton — Sowing Season and Bollworm Management
06_tomato_advisory.txt	Tomato (<i>Solanum lycopersicum</i>)	Tomato — Transplanting Season and Pest/Disease Advisory
07_maize_fall_armyworm.txt	Maize (<i>Zea mays</i>)	Maize — Sowing Season and Fall Armyworm Advisory
08_groundnut_advisory.txt	Groundnut / Peanut (<i>Arachis hypogaea</i>)	Groundnut — Sowing Season and Pest Advisory
09_chilli_advisory.txt	Chilli / Pepper (<i>Capsicum annuum</i>)	Chilli — Season, Thrips, Mites and Disease Advisory
10_sugarcane_advisory.txt	Sugarcane (<i>Saccharum officinarum</i>)	Sugarcane — Planting Season and Early Shoot Borer Advisory
11_ipm_and_safe_pesticide_use.txt	(general)	Integrated Pest Management (IPM) and Safe Pesticide Use — General Advisory
12_seasonal_crop_calendar.txt	(general)	Seasonal Crop Calendar — Kharif, Rabi and Zaid Advisory
13_soybean_advisory.txt	Soybean (<i>Glycine max</i>)	Soybean — Sowing Season and Pest Advisory
14_mustard_advisory.txt	Mustard / Rapeseed (<i>Brassica juncea</i>)	Mustard — Sowing Season and Aphid Advisory
15_pigeonpea_advisory.txt	Pigeon Pea / Red Gram (<i>Cajanus cajan</i>)	Pigeon Pea (Arhar/Tur) — Sowing Season and Pod Borer Advisory
16_chickpea_advisory.txt	Chickpea / Bengal Gram (<i>Cicer arietinum</i>)	Chickpea (Gram) — Sowing Season and Pod Borer

		Advisory
17_onion_advisory.txt	Onion (Allium cepa)	Onion — Season, Thrips and Purple Blotch Advisory
18_potato_advisory.txt	Potato (Solanum tuberosum)	Potato — Planting Season and Late Blight Advisory
19_brinjal_advisory.txt	Brinjal / Eggplant (Solanum melongena)	Brinjal (Eggplant) — Season and Shoot & Fruit Borer Advisory
20_banana_advisory.txt	Banana (Musa)	Banana — Planting Season and Pest/Disease Advisory

6. Demonstration - The Five Required Queries

#	Query	Requirement	Result
1	How do I control stem borer in paddy?	Answerable - pest control	Retrieves paddy pest doc; Trichogramma + threshold spray, cited [C4][C7].
2	When is the right time to sow wheat?	Answerable - sowing season	Rewrite adds 'season sowing'; wheat doc; November (Rabi), cited [C8].
3	Small insects are making holes in my tomato fruits, what should I do?	Answerable - non-technical phrasing	Rewrite adds 'pest borer feeding damage'; still retrieves tomato fruit borer, cited [C17][C18].
4	Compare the sowing seasons of paddy and wheat.	Multiple chunks / documents	Retrieves paddy AND wheat docs; contrasts Kharif vs Rabi, cited [C1][C8].
5	What is today's gold price in the market?	Out-of-scope	Score 0.000 < 0.06 -> agent refuses instead of hallucinating.

Captured program output (lab/DEMO_OUTPUT.txt):

```
=====
FARMER ADVISORY AGENT - RAG pipeline
=====
Corpus documents : 20 (farmer_corpus)
Chunks indexed   : 66 (size=110 words, overlap=25 words)
Vocabulary terms : 1032
Embedding model  : from-scratch TF-IDF (sublinear tf, smoothed idf)
Vector store     : in-memory cosine index | top-k = 3
LLM generator    : nvidia/nemotron-nano-9b-v2:free
=====

### 1. Answerable - PEST CONTROL
QUERY : How do I control stem borer in paddy?
TOP MATCH SCORE : 0.294 (floor to answer = 0.06)
RETRIEVED CHUNKS (grounding):
  [C4] 02_paddy_pest_control.txt score=0.294
        TITLE: Paddy (Rice) – Pest and Insect Control Advisory CROP: Paddy / Rice SOURCE: Compiled crop advisory (public
agricultural extension style) MAJOR P...
  [C7] 02_paddy_pest_control.txt score=0.282
        "white ear head" indicate stem borer. - "Hopper burn" (brown drying patches) indicates brown planthopper. -
Trichogramma release is the main biologica...
  [C1] 01_paddy_sowing_season.txt score=0.229
        TITLE: Paddy (Rice) – Sowing Season and Nursery Advisory CROP: Paddy / Rice (Oryza sativa) SOURCE: Compiled crop
advisory (public agricultural extensi...
ANSWER (LLM):
To control stem borer in paddy:
1. **Clip and destroy egg masses** on seedling tips before transplanting [C4].
2. **Release Trichogramma japonicum** egg parasitoids at 50,000 per hectare weekly [C4, C7].
3. If damage exceeds one egg mass per square metre, apply **cartap hydrochloride 4G or chlorantraniliprole** as per label
instructions [C4].

These steps prioritize biological control and targeted chemical use if needed.
-----

### 2. Answerable - SOWING SEASON
QUERY : When is the right time to sow wheat?
REWRITE (agent) : + season sowing
TOP MATCH SCORE : 0.292 (floor to answer = 0.06)
RETRIEVED CHUNKS (grounding):
  [C8] 03_wheat_sowing_season.txt score=0.292
        TITLE: Wheat – Sowing Season and Seed Rate Advisory CROP: Wheat (Triticum aestivum) SOURCE: Compiled crop advisory
(public agricultural extension styl...
  [C39] 12_seasonal_crop_calendar.txt score=0.200
        sugarcane (as a long-duration crop). 2. RABI (winter season) Sowing: October to December. Harvest: March to April.
```

```

These are cool-season crops grown o...
[C40] 12_seasonal_crop_calendar.txt      score=0.197
    SEASON SELECTION MATTERS Choosing the right season aligns the crop's water and temperature needs with the natural
climate, reduces pest and disease pr...
ANSWER (LLM):
The right time to sow wheat depends on irrigation:
- **Irrigated wheat**: Sow between the first week of November and the last week of November [C8].
- **Rain-fed wheat**: Sow in the second fortnight of October [C8].

Late sowing up to the third week of December is possible but reduces yield by 1-1.5% per day after mid-November [C8].
Choosing the correct season aligns with climate needs and improves yield [C39, C40].
-----

### 3. Answerable - SIMPLE NON-TECHNICAL PHRASING
QUERY : Small insects are making holes in my tomato fruits, what should I do?
REWRITE (agent) : + pest borer feeding damage
TOP MATCH SCORE : 0.237 (floor to answer = 0.06)
RETRIEVED CHUNKS (grounding):
[C17] 06_tomato_advisory.txt              score=0.237
    TITLE: Tomato – Transplanting Season and Pest/Disease Advisory CROP: Tomato (Solanum lycopersicum) SOURCE:
Compiled crop advisory (public agricultural...
[C18] 06_tomato_advisory.txt              score=0.233
    at a spacing of 60 cm between rows and 45 cm between plants. FRUIT BORER (Helicoverpa) Symptoms: round holes bored
into green and ripening fruits; the...
[C20] 06_tomato_advisory.txt              score=0.150
    the whitefly vector with yellow sticky traps and recommended insecticides, and remove infected plants early. KEY
POINTS - Rabi tomato: nursery Septemb...
ANSWER (LLM):
To address small insects boring holes in your tomato fruits, follow these steps:
1. **Install pheromone traps** to monitor and reduce Fruit Borer moths [C17].
2. **Hand-pick and destroy** affected fruits and larvae [C17].
3. **Spray Bacillus thuringiensis (Bt)** or a recommended insecticide when small larvae are present [C17].
4. **Set up bird perches** to help control the pest naturally [C17].

These measures target the Fruit Borer (Helicoverpa), which is the most damaging pest for this symptom [C18].
-----

### 4. Multi-chunk - needs MORE THAN ONE document
QUERY : Compare the sowing seasons of paddy and wheat.
TOP MATCH SCORE : 0.303 (floor to answer = 0.06)
RETRIEVED CHUNKS (grounding):
[C1] 01_paddy_sowing_season.txt           score=0.303
    TITLE: Paddy (Rice) – Sowing Season and Nursery Advisory CROP: Paddy / Rice (Oryza sativa) SOURCE: Compiled crop
advisory (public agricultural extensi...
[C38] 12_seasonal_crop_calendar.txt       score=0.209
    TITLE: Seasonal Crop Calendar – Kharif, Rabi and Zaid Advisory TOPIC: Cropping seasons of India SOURCE: Compiled
crop advisory (public agricultural ex...
[C8] 03_wheat_sowing_season.txt           score=0.205
    TITLE: Wheat – Sowing Season and Seed Rate Advisory CROP: Wheat (Triticum aestivum) SOURCE: Compiled crop advisory
(public agricultural extension styl...
ANSWER (LLM):
Paddy is sown in three seasons: Kharif (June-July), Rabi (November-December), and Zaid (January-February) [C1]. Wheat is
primarily a Rabi crop, sown from early November to mid-December [C8]. Paddy has more flexible sowing windows across
seasons, while wheat is concentrated in Rabi with limited rain-fed options in October.
-----

### 5. Out-of-scope - system must REFUSE, not hallucinate
QUERY : What is today's gold price in the market?
TOP MATCH SCORE : 0.000 (floor to answer = 0.06)
ROUTE : OUT-OF-SCOPE -> refusing to answer
ANSWER:
I could not find this in the crop advisory documents. This system only answers questions about the crop advisories in its
corpus (sowing seasons, pest and disease control, IPM).
-----

```

7. How to Run

```

# run the 5 required demo queries (with LLM answers)
python lab/q13_farmer_rag.py

# ask any single question

```

```
python lab/q13_farmer_rag.py "how do I sow cotton and stop bollworm"
```

```
# offline / deterministic: extractive answers only, no LLM calls  
FARMER_USE_LLM=0 python lab/q13_farmer_rag.py
```

8. Conclusion

The Farmer Advisory Agent implements every stage of a RAG pipeline over a 20-document crop-advisory corpus and demonstrates the required behaviours: grounded, cited answers for pest-control and sowing-season questions, correct retrieval for a non-technical farmer query through agent-side query rewriting, a multi-document comparison answer, and a safe refusal for an out-of-scope question. The design favours transparency and reliability - a from-scratch TF-IDF embedder and in-memory cosine store make the retrieval fully inspectable, while the score-floor router and grounding prompt prevent hallucination.

Appendix A - Full Source Code (lab/q13_farmer_rag.py)

```
# -*- coding: utf-8 -*-
"""
Question 13 - Farmer Advisory Agent (RAG over public crop advisory documents)
=====

A complete, self-contained Retrieval-Augmented-Generation pipeline:

    document loading -> chunking -> embedding -> vector-store indexing
    -> retrieval -> AGENT (route + query-rewrite) -> grounded answer (cited)

Design goals for this assessment:
* Answers are GROUNDED - the system prints the exact retrieved chunk(s) it used
  and cites them inline as [C1], [C2], ...
* AGENT behaviour is explicit: (1) a query-understanding step that rewrites plain
  farmer language into technical terms, and (2) a routing/refusal step that says
  "I cannot answer" for out-of-scope questions instead of hallucinating.
* No heavy dependencies. The embedding model is a from-scratch TF-IDF vectoriser
  and the vector store is an in-memory cosine index, so the demo always runs.
* The LLM (same OpenAI-compatible provider as the main app, via lab/llm.py) writes
  the final fluent answer. If the LLM is unavailable/rate-limited, the system falls
  back to an EXTRACTIVE answer built from the retrieved chunks - still fully
  grounded and cited - so the pipeline never breaks during the demo.

Run:
python lab/q13_farmer_rag.py          # run the 5 required demo queries
python lab/q13_farmer_rag.py "your question here" # ask one question
FARMER_USE_LLM=0 python lab/q13_farmer_rag.py      # extractive only (offline)
"""

import os
import re
import sys
import math
from pathlib import Path
from collections import Counter

# --- make stdout UTF-8 (Windows terminals default to cp1252) -----
for _s in (sys.stdout, sys.stderr):
    try:
        _s.reconfigure(encoding="utf-8")
    except Exception: # noqa: BLE001
        pass

# --- reuse the project's shared LLM helper (same .env / key / model) -----
sys.path.insert(0, str(Path(__file__).resolve().parent))
try:
    from llm import chat as _llm_chat, MODEL as _LLM_MODEL
except Exception: # noqa: BLE001 - helper is optional; extractive mode still works
    _llm_chat, _LLM_MODEL = None, "(none)"

USE_LLM = os.getenv("FARMER_USE_LLM", "1") == "1" and _llm_chat is not None

# =====
# 0. DESIGN CHOICES (also documented in lab/README_Q13_FARMER_RAG.md)
# =====
CORPUS_DIR = Path(__file__).resolve().parent / "farmer_corpus"
CHUNK_WORDS = 110 # target chunk size in words (~750 characters)
CHUNK_OVERLAP = 25 # overlap in words between consecutive chunks
TOP_K = 3 # number of chunks retrieved per query
SCORE_FLOOR = 0.06 # min cosine similarity; below this -> "out of scope"

STOPWORDS = set("""
a an and are as at be by for from has have how i in is it its of on or that the to
was were will with what when where which who whom this these those you your my me we
do does did can could should would may might must shall about into over under out
""").split())

# =====
# 1. DOCUMENT LOADING
# =====
def load_documents(folder: Path):
    """Load every .txt advisory file. Returns list of {source, title, text}."""
    docs = []
    for path in sorted(folder.glob("*.txt")):
```



```

text = path.read_text(encoding="utf-8").strip()
title = ""
for line in text.splitlines():
    if line.upper().startswith("TITLE:"):
        title = line.split(":", 1)[1].strip()
        break
docs.append({"source": path.name, "title": title or path.stem, "text": text})
return docs

# =====
# 2. CHUNKING (word-based sliding window with overlap)
# =====
def chunk_document(doc):
    """Split one document into overlapping word windows."""
    words = doc["text"].split()
    step = CHUNK_WORDS - CHUNK_OVERLAP
    chunks = []
    if not words:
        return chunks
    for start in range(0, len(words), step):
        window = words[start:start + CHUNK_WORDS]
        if not window:
            break
        chunks.append({
            "source": doc["source"],
            "title": doc["title"],
            "text": " ".join(window),
        })
        if start + CHUNK_WORDS >= len(words):
            break
    return chunks

# =====
# 3. EMBEDDING MODEL (from-scratch TF-IDF vectoriser)
# =====
def tokenize(text):
    return [t for t in re.findall(r"[a-z0-9]+", text.lower())
            if t not in STOPWORDS and len(t) > 1]

class TfidfEmbedder:
    """Learns IDF from the corpus, then embeds any text into an L2-normalised
    sparse TF-IDF vector (dict term -> weight). This is our 'embedding model'."""

    def __init__(self):
        self.idf = {}

    def fit(self, list_of_token_lists):
        n = len(list_of_token_lists)
        df = Counter()
        for toks in list_of_token_lists:
            for term in set(toks):
                df[term] += 1
        # smoothed idf so no division by zero and rare terms weigh more
        self.idf = {t: math.log((n + 1) / (c + 1)) + 1.0 for t, c in df.items()}
        return self

    def embed(self, tokens):
        tf = Counter(tokens)
        vec = {}
        for term, count in tf.items():
            if term in self.idf:
                # ignore out-of-vocab terms
                vec[term] = (1.0 + math.log(count)) * self.idf[term] # sublinear tf
        norm = math.sqrt(sum(w * w for w in vec.values()))
        if norm:
            for term in vec:
                vec[term] /= norm
        return vec

    def cosine(a, b):
        """Dot product of two L2-normalised sparse vectors = cosine similarity."""
        if len(a) > len(b):
            a, b = b, a
        return sum(w * b.get(t, 0.0) for t, w in a.items())

```

```

# =====
# 4. VECTOR STORE (in-memory cosine index)
# =====
class VectorStore:
    def __init__(self, embedder):
        self.embedder = embedder
        self.chunks = [] # list of chunk dicts (with 'id', 'vector')

    def index(self, chunks):
        token_lists = [tokenize(c["text"]) for c in chunks]
        self.embedder.fit(token_lists)
        for i, (c, toks) in enumerate(zip(chunks, token_lists), start=1):
            c = dict(c)
            c["id"] = "C%d" % i
            c["vector"] = self.embedder.embed(toks)
            self.chunks.append(c)
        return self

    def search(self, query_text, top_k=TOP_K):
        qvec = self.embedder.embed(tokenize(query_text))
        scored = [(cosine(qvec, c["vector"]), c) for c in self.chunks]
        scored.sort(key=lambda x: x[0], reverse=True)
        return scored[:top_k]

# =====
# 5. AGENT LAYER (query understanding + routing + grounded generation)
# =====

# Plain-language -> technical term expansion. This is the agent's "tool" that lets
# a non-technical farmer question still retrieve the right technical chunk.
SYNONYMS = {
    "bug": "pest insect", "bugs": "pest insect", "insect": "pest",
    "insects": "pest", "worm": "borer larva caterpillar",
    "worms": "borer larva caterpillar", "caterpillar": "borer larva",
    "holes": "borer feeding damage", "hole": "borer feeding damage",
    "eating": "feeding damage pest", "eaten": "feeding damage pest",
    "chewed": "feeding damage", "spots": "leaf spot disease",
    "rotting": "rot disease", "rot": "rot disease",
    "medicine": "insecticide pesticide spray", "spray": "insecticide pesticide",
    "chemical": "insecticide pesticide", "plant": "sow crop",
    "planting": "sowing season", "grow": "sow season",
    "growing": "sowing season", "sow": "sowing season",
    "time": "season", "month": "season sowing", "when": "sowing season",
    "yellow": "yellowing disease", "dying": "wilting dead heart disease",
    "curling": "leaf curl", "small": "", "making": "",
}

def rewrite_query(query):
    """AGENT STEP 1 - query understanding: expand plain farmer language into the
    technical vocabulary used in the advisories. Returns (expanded_text, added)."""
    added = []
    for tok in tokenize(query):
        if tok in SYNONYMS and SYNONYMS[tok]:
            for extra in SYNONYMS[tok].split():
                if extra not in added:
                    added.append(extra)
    expanded = query if not added else query + " " + " ".join(added)
    return expanded, added

def build_context(hits):
    """Format retrieved chunks as a numbered, citable context block."""
    lines = []
    for score, c in hits:
        snippet = re.sub(r"\s+", " ", c["text"]).strip()
        lines.append("[%s] (source: %s | score %.3f)\n%s" %
                     (c["id"], c["source"], score, snippet))
    return "\n\n".join(lines)

PROMPT_SYSTEM = (
    "You are a Farmer Advisory Agent. Answer the farmer's question ONLY using the "

```

```

"numbered context passages provided. Cite every passage you use inline with its "
>tag, e.g. [C1], [C2]. Give short, practical, plain-language advice. If the answer "
"is not present in the context, reply exactly: "
"I could not find this in the crop advisory documents.""
)

def extractive_answer(query_expanded, hits):
    """Fallback (and offline) generator: pick the sentences from the retrieved
    chunks that best overlap the query, and cite their chunk id. Fully grounded."""
    qterms = set(tokenize(query_expanded))
    scored_sentences = []
    for _score, c in hits:
        for sent in re.split(r"(?<=[.!?])\s+", c["text"]):
            st = set(tokenize(sent))
            overlap = len(qterms & st)
            if overlap:
                scored_sentences.append((overlap, sent.strip(), c["id"]))
    scored_sentences.sort(key=lambda x: x[0], reverse=True)
    if not scored_sentences:
        top = hits[0][1]
        return "%s [%s]" % (top["text"].split(". ")[0].strip(), top["id"])
    seen, out = set(), []
    for _ov, sent, cid in scored_sentences:
        key = sent[:60]
        if key in seen:
            continue
        seen.add(key)
        out.append("- %s [%s]" % (sent.rstrip("."), cid))
        if len(out) >= 4:
            break
    return "\n".join(out)

def generate(query, query_expanded, hits):
    """Produce the final grounded answer. Tries the LLM, falls back to extractive."""
    context = build_context(hits)
    if USE_LLM:
        prompt = ("Context passages:\n%s\n\nFarmer's question: %s\n\n"
                  "Grounded answer with citations:" % (context, query))
        try:
            return _llm_chat(prompt, temperature=0.2, system=PROMPT_SYSTEM), "LLM"
        except SystemExit as e:
            # llm.py raises SystemExit on missing key / HTTP error
            print(" [LLM unavailable: %s -> using extractive fallback]" % e)
        except Exception as e:
            # noqa: BLE001 - network, timeout, rate-limit, etc.
            print(" [LLM error: %s -> using extractive fallback]" % e)
    return extractive_answer(query_expanded, hits), "extractive"

def answer_question(store, query, show_chunks=True):
    """The full agent pipeline for a single question."""
    # AGENT STEP 1 - understand & rewrite the query
    expanded, added = rewrite_query(query)

    # AGENT STEP 2 - retrieve from the vector store
    hits = store.search(expanded, TOP_K)
    best = hits[0][0] if hits else 0.0

    print("QUERY : %s" % query)
    if added:
        print("REWRITE (agent) : + %s" % " ".join(added))
    print("TOP MATCH SCORE : %.3f (floor to answer = %.2f)" % (best, SCORE_FLOOR))

    # AGENT STEP 3 - route: refuse if nothing is relevant (no hallucination)
    if best < SCORE_FLOOR:
        print("ROUTE : OUT-OF-SCOPE -> refusing to answer")
        print("ANSWER:\nI could not find this in the crop advisory documents. "
              "This system only answers questions about the crop advisories in its "
              "corpus (sowing seasons, pest and disease control, IPM).")
        print("-" * 78)
        return

    if show_chunks:
        print("RETRIEVED CHUNKS (grounding):")
        for score, c in hits:
            snippet = re.sub(r"\s+", " ", c["text"]).strip()
            print(" [%s] %-34s score=%.3f" % (c["id"], c["source"], score))
            print(" %s..." % snippet[:150])

```

```

# AGENT STEP 4 - grounded generation with citations
ans, mode = generate(query, expanded, hits)
print("ANSWER (%s):\n%s" % (mode, ans))
print("-" * 78)

# =====
# 6. BUILD THE PIPELINE + DEMO
# =====
def build_store():
    docs = load_documents(CORPUS_DIR)
    all_chunks = []
    for d in docs:
        all_chunks.extend(chunk_document(d))
    store = VectorStore(TfidfEmbedder()).index(all_chunks)
    return docs, all_chunks, store

DEMO_QUERIES = [
    ("1. Answerable - PEST CONTROL",
     "How do I control stem borer in paddy?"),
    ("2. Answerable - SOWING SEASON",
     "When is the right time to sow wheat?"),
    ("3. Answerable - SIMPLE NON-TECHNICAL PHRASING",
     "Small insects are making holes in my tomato fruits, what should I do?"),
    ("4. Multi-chunk - needs MORE THAN ONE document",
     "Compare the sowing seasons of paddy and wheat."),
    ("5. Out-of-scope - system must REFUSE, not hallucinate",
     "What is today's gold price in the market?"),
]

def main():
    docs, chunks, store = build_store()

    print("=" * 78)
    print("FARMER ADVISORY AGENT - RAG pipeline")
    print("=" * 78)
    print("Corpus documents : %d (%s)" % (len(docs), CORPUS_DIR.name))
    print("Chunks indexed : %d (size=%d words, overlap=%d words)"
          % (len(chunks), CHUNK_WORDS, CHUNK_OVERLAP))
    print("Vocabulary terms : %d" % len(store.embedder.idf))
    print("Embedding model : from-scratch TF-IDF (sublinear tf, smoothed idf)")
    print("Vector store : in-memory cosine index | top-k = %d" % TOP_K)
    print("LLM generator : %s" % (_LLM_MODEL if USE_LLM else "OFF (extractive)"))
    print("=" * 78)

    # single-question mode
    if len(sys.argv) > 1:
        print()
        answer_question(store, " ".join(sys.argv[1:]))
        return

    # demo mode - the five required queries
    for label, q in DEMO_QUERIES:
        print("\n### %s" % label)
        answer_question(store, q)

if __name__ == "__main__":
    main()

```